

Despliegue de Aurora Financiera en EC2 (Apache + Gunicorn + HTTPS)

Guía paso a paso para desplegar la app **Django** en una instancia EC2 con Apache2 como *reverse proxy* hacia Gunicorn, estáticos servidos desde `/var/www/html/static`, y HTTPS con Let's Encrypt (Certbot) para `sruedadev.com`.

Arquitectura final:



- Código Django: `/var/www/aurora`
- Estáticos públicos: `/var/www/html/static`

⚠ 0. Seguridad ANTES de empezar

Estos archivos **NUNCA** deben subir al servidor ni a git (ya están en `.gitignore`):

- `.env` → contiene `SECRET_KEY` y credenciales.
- `n8n-bedrock-ocr_accessKeys.csv` → **contiene claves AWS**. Si alguna vez se expuso, **rótala/desactívala en la consola de AWS IAM** por precaución.

En producción se crea un `.env` nuevo directamente en la EC2 (paso 5). `SECRET_KEY` de producción debe ser **distinta** a la local.

1. Subir el código a GitHub (desde tu máquina local)

El repo ya está inicializado y con el commit inicial. Créalo en GitHub y súbelo (ya tienes `gh` autenticado como `srueda123`):

```
cd /home/srueda/Documentos/linux/aurora-financiera
gh repo create aurora-financiera --private --source=. --remote=origin --push
```

Esto crea el repo **privado**, añade el remoto `origin` y sube la rama `main`. Verifica: `git remote -v`

El `.gitignore` ya protege `.env` y el CSV de claves AWS: **no se suben**.

1-bis. Credenciales en la EC2 para clonar (Deploy Key SSH — recomendado)

Como el repo es **privado**, la EC2 necesita credenciales para clonarlo. La forma más segura es una **Deploy Key**: un par de llaves SSH exclusivo de este repo y de **solo lectura** (si se filtra, no compromete tu cuenta ni otros repos).

En la **EC2**, genera la llave:

```
ssh-keygen -t ed25519 -C "ec2-aurora-deploy" -f ~/.ssh/aurora_deploy -N ""  
cat ~/.ssh/aurora_deploy.pub          # copia TODO lo que imprime
```

En **GitHub** (navegador): repo **aurora-financiera** → **Settings** → **Deploy keys** → **Add deploy key** → pega la clave pública, título **ec2-aurora**, **NO** marques "Allow write access" → **Add key**.

En la **EC2**, indica a git que use esa llave para GitHub:

```
cat >> ~/.ssh/config <<'EOF'  
Host github-aurora  
    HostName github.com  
    User git  
    IdentityFile ~/.ssh/aurora_deploy  
    IdentitiesOnly yes  
EOF  
chmod 600 ~/.ssh/config
```

Prueba y clona:

```
ssh -T git@github-aurora          # debe saludarte con tu usuario  
git clone git@github-aurora:srueda123/aurora-financiera.git /tmp/aurora-src
```

Alternativa (PAT por HTTPS): en vez de la Deploy Key, en GitHub crea un *Fine-grained token* de solo lectura (Settings → Developer settings → Personal access tokens) con acceso a este repo, y clona con:
`git clone https://USUARIO:TOKEN@github.com/srueda123/aurora-financiera.git`
La Deploy Key es preferible en servidores.

1-ter. Entrar a la instancia

```
ssh -i ~/.ssh/TU_LLAVE.pem ubuntu@IP_EC2
```

(Ubuntu AMI usa el usuario **ubuntu**; Amazon Linux usa **ec2-user**. Usa la Elastic IP si tienes.)

2. Instalar dependencias del sistema (en la EC2)

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip apache2 \
    libapache2-mod-proxy-html

# Módulos de Apache necesarios para el proxy y el header HTTPS:
sudo a2enmod proxy proxy_http headers ssl rewrite
sudo systemctl restart apache2
```

3. Clonar el código en `/var/www/aurora`

Clonamos directamente ahí (dueño `ubuntu`) para que luego `git pull` actualice sin fricción:

```
sudo mkdir -p /var/www/aurora /var/www/html/static
sudo chown ubuntu:ubuntu /var/www/aurora /var/www/html/static
git clone git@github-aurora:srueda123/aurora-financiera.git /var/www/aurora
```

4. Entorno virtual + dependencias de Python

```
cd /var/www/aurora
python3 -m venv env
./env/bin/pip install --upgrade pip
./env/bin/pip install -r requirements.txt
```

Si usarás **SQLite** (opción simple) puedes quitar `psycopg2-binary` del `requirements.txt`. Si usarás **PostgreSQL/Supabase**, déjalo.

5. Crear el `.env` de producción

```
cp /var/www/aurora/.env.produccion.example /var/www/aurora/.env
nano /var/www/aurora/.env
```

Rellena como mínimo:

- `SECRET_KEY` → génerala:

```
./env/bin/python -c "import secrets;print(secrets.token_urlsafe(50))"
```

- `DEBUG=False`
 - `ALLOWED_HOSTS=sruedadev.com,www.sruedadev.com,127.0.0.1`
 - `CSRF_TRUSTED_ORIGINS=https://sruedadev.com,https://www.sruedadev.com`
 - `STATIC_ROOT=/var/www/html/static`
 - Base de datos (SQLite o Postgres, ver la plantilla).
 - `N8N_WEBHOOK_URL` → usa la URL de **producción** (`/webhook/aura-web`, no la de test).
-

6. Migraciones + recopilar estáticos

```
cd /var/www/aurora
./env/bin/python manage.py migrate
./env/bin/python manage.py collectstatic --noinput # ->
/var/www/html/static
```

(Opcional) crea un superusuario para `/admin/`:

```
./env/bin/python manage.py createsuperuser
```

7. Permisos

El código lo maneja `ubuntu` (dueño del clon y de la Deploy Key) y Gunicorn corre como `ubuntu`. Apache solo necesita **leer** los estáticos:

```
chmod 600 /var/www/aurora/.env # solo tu usuario lo lee
chmod -R a+rX /var/www/html/static # Apache (www-data) puede leerlos
```

8. Servicio Gunicorn (systemd)

```
sudo cp /var/www/aurora/deploy/gunicorn.service \
/etc/systemd/system/gunicorn-aurora.service

sudo systemctl daemon-reload
sudo systemctl enable --now gunicorn-aurora
sudo systemctl status gunicorn-aurora # debe verse "active (running)"
```

Prueba local de que Django responde:

```
curl -I http://127.0.0.1:8001/ # debe devolver HTTP 200/302
```

9. Configurar Apache (VirtualHost)

```
sudo cp /var/www/aurora/deploy/aurora-apache.conf \
    /etc/apache2/sites-available/aurora.conf

sudo a2ensite aurora.conf
sudo a2dissite 000-default.conf      # desactiva el sitio por defecto
sudo apache2ctl configtest          # debe decir "Syntax OK"
sudo systemctl reload apache2
```

Comprueba en el navegador <http://srudedev.com> — ya debería cargar la landing con estilos e imágenes.

10. HTTPS con Certbot (Let's Encrypt)

```
sudo apt install -y certbot python3-certbot-apache
sudo certbot --apache -d srudedev.com -d www.srudedev.com
```

Elige **redirección automática HTTP → HTTPS** cuando lo pregunte. Certbot crea el bloque `<VirtualHost *:443>` con el certificado.

10.1 Ajuste importante tras Certbot

Edita el vhost SSL que generó Certbot y asegúrate de que el header enviado a Django sea **https** (para que valide CSRF del chatbot y genere URLs seguras):

```
sudo nano /etc/apache2/sites-available/aurora-le-ssl.conf
```

Dentro del `<VirtualHost *:443>`, deja/añade:

```
RequestHeader set X-Forwarded-Proto "https"
```

Luego:

```
sudo apache2ctl configtest && sudo systemctl reload apache2
```

Renovación automática (ya viene por defecto). Verifícala:

```
sudo certbot renew --dry-run
```

11. Verificación final

- <https://srudedev.com> carga con candado ✓
- Imágenes y CSS cargan (vienen de `/static/`) ✓
- El chatbot **Aurorita** responde (prueba escribir un mensaje) — esto valida que `POST /api/chat/` → funciona con CSRF/HTTPS ✓
- <https://srudedev.com/admin/> abre el admin ✓

12. Actualizaciones futuras (redesploy con git)

En tu máquina, haz commit y push de los cambios:

```
git add -A && git commit -m "... " && git push
```

En la EC2, baja los cambios y recarga:

```
cd /var/www/aurora
sudo -u www-data git pull
sudo ./env/bin/python manage.py migrate
sudo ./env/bin/python manage.py collectstatic --noinput
sudo systemctl restart gunicorn-aurora
```

`sudo -u www-data git pull` mantiene los permisos correctos. Como el paso 3 clonó el repo en `/var/www/aurora`, esa carpeta ya es un repo git conectado a tu `origin`.

13. Solución de problemas

Síntoma	Dónde mirar / Causa probable
Error 500	<code>sudo journalctl -u gunicorn-aurora -n 50</code> — falta una var en <code>.env</code> o dependencia
CSS/imágenes rotas	¿Corriste <code>collectstatic</code> ? ¿ <code>STATIC_ROOT=/var/www/html/static</code> ? ¿permisos <code>www-data</code> ?
502 Bad Gateway	Gunicorn caído: <code>sudo systemctl status gunicorn-aurora</code>
Chatbot da 403	Falta <code>CSRF_TRUSTED_ORIGINS</code> o el header <code>X-Forwarded-Proto https</code> en el <code>vhost 443</code>
<code>DisallowedHost</code>	Añade el dominio a <code>ALLOWED_HOSTS</code> en <code>.env</code>

Síntoma	Dónde mirar / Causa probable
Chatbot no conecta	<code>N8N_WEBHOOK_URL</code> apunta a <code>/webhook-test/</code> (solo dev). Usa <code>/webhook/aura-web</code>
Logs de Apache	<code>sudo tail -f /var/log/apache2/aurora_error.log</code>

Resumen de puertos / rutas

Elemento	Valor
Código Django	<code>/var/www/aurora</code>
Virtualenv	<code>/var/www/aurora/env</code>
Estáticos	<code>/var/www/html/static</code>
Gunicorn	<code>127.0.0.1:8001</code> (interno)
Apache	<code>:80</code> → redirige a <code>:443</code>
Servicio	<code>gunicorn-aurora.service</code>